1 siege
- 1.1 安装
  - 1.1.1 mac
  - 1.1.2 linux 安装
- 1.2 参数
- 1.3 使用
  - 1.3.1 获取 Header 信息
  - 1.3.2 get
  - 1.3.3 post
- 1.4 输出解释

2 locust
- 2.1 安装
- 2.2 重要参数
- 2.3 使用
  - 2.3.1 UI 启动
  - 2.3.2 命令行启动
  - 2.3.3 py文件启动
- 2.4 web输出解释
  - 2.4.1 STATISTICS
  - 2.4.2 CHARTS
- 2.5 搜索场景

3 evalscope
- 3.1 安装
- 3.2 重要参数
- 3.3 使用
- 3.4 输出解释
- 3.5 解析代码

1 siege

1.1 安装

1.1.1 mac

- 使用 brew 安装

```
brew install siege
```

- 版本（4.16 为最新版本）

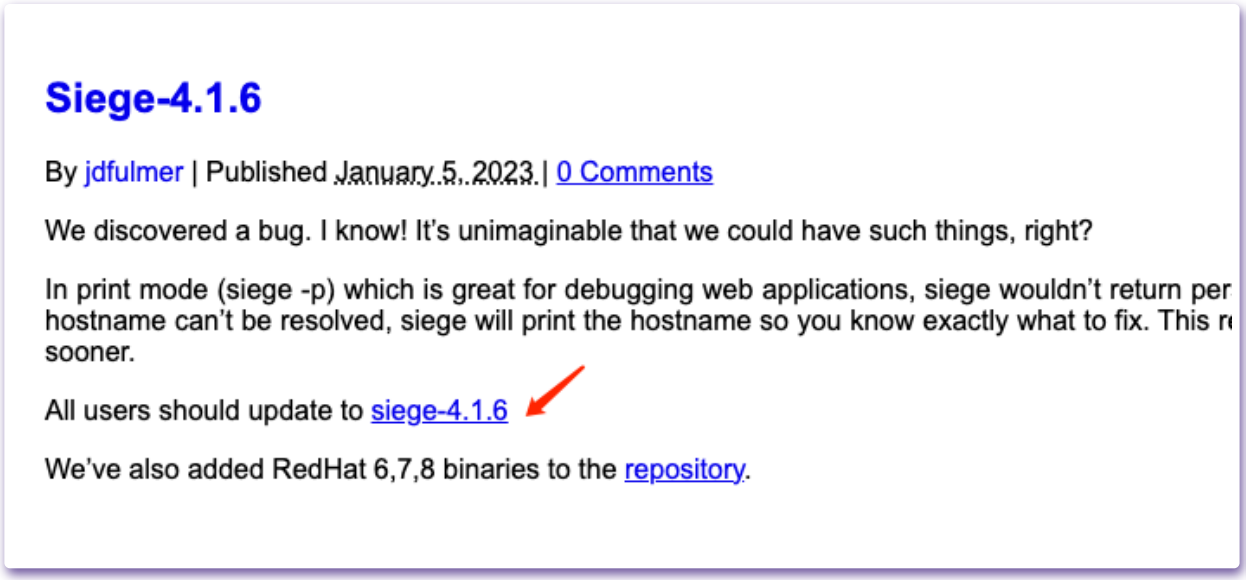
```
siege -V

SIEGE 4.1.6
Copyright (C) 2023 by Jeffrey Fulmer, et al.
This is free software; see the source for copying conditions.
There is NO warranty; not even for MERCHANTABILITY or FITNESS
FOR A PARTICULAR PURPOSE.
```

1.1.2 linux 安装

- **下载地址**

> https://www.joedog.org/



Siege-4.1.6

By jdfulmer | Published January 5, 2023 | 0 Comments

We discovered a bug. I know! It's unimaginable that we could have such things, right?

In print mode (siege -p) which is great for debugging web applications, siege wouldn't return per
hostname can't be resolved, siege will print the hostname so you know exactly what to fix. This r
sooner.

All users should update to siege-4.1.6

We've also added RedHat 6,7,8 binaries to the repository.

- 解压

```
tar -zxvf siege-4.1.6.tar.gz
```

- 编译安装

```
cd siege-4.1.6
./configure
make && make install
```

- 查看命令

```
which siege
siege -V
```

1.2 参数

- 查看参数

```
siege -h
```

- 参数说明

参数	参数说明
-V, -version	版本，打印版本号。
-h, --help	帮助，打印此部分。
-C, --config	配具，显示当前配置。
-v, -verbose	详细，将通知打印到屏幕。
-q,--quiet	安静关闭冗余并抑制输出。
-g, --get	获取、下拉HTTP头并显示、适合应用程序调试
-p, -print	打印，就像只获取一样，它打印整个页面。
-c, --concurrent=NUM	并发用户，默认值为10
-r, --reps=NUM	重复次数，运行测试的次数。
-t, --time=NUMm	定时测试，其中“m”是修饰符S、m或H ; ex:–时间=1H，一小时试验。
-d, --delay=NUM	时间延迟，每个请求前的随机延迟 在 1-num秒
-b, --benchmark	基准测试；请求之间没有延迟。
-i, --internet	互联网用户模拟，随机点击网址。
-f, -file=FILE	文件，选择特定的URL文件。
$-\mathrm{R}, -\mathrm{rc} = \mathrm{FILE}$	RC，指定一个siegerc文件
-1,--log[=FILE]	记录到文件。如果未指定文件，则使用默认值：前缀/var/seake.log
-m, -mark="text"	标记，用字符串标记日志文件，介于.001和NUM之间（不计入统计数据）
-H, -header="text"	向清求添加标头 (可以是多个)
-A, -user-agent="text"	在请求中设置用户代理
-T, -content-type="text"	设惠请求中的内容类型
-j, -json-output	JSON输出，将最终统计数据作为JSON打印到标准输出
-no-parser	没有解析器，请关闭HTML页面解析器
-no-follow	不跟随，不跟随HTTP重定向

1.3 使用

1.3.1 获取 Header 信息

- 获取指定的 url 的 header 信息

```
siege -g http://www.baidu.com
```

1.3.2 get

- 注意，-r 指定重复次数，-t 指定时间，两个不可同时使用，否则只会使用-t
- 并发 100，重复一次

```
siege -c 100 -r 1 http://8.210.74.55:8002
```

```
Transactions:            100 hits
Availability:         100.00 %
Elapsed time:           0.58 secs
Data transferred:         0.00 MB
Response time:            0.42 secs
Transaction rate:       172.41 trans/sec
Throughput:           0.00 MB/sec
Concurrency:           72.64
Successful transactions:         100
Failed transactions:             0
Longest transaction:          0.58
Shortest transaction:         0.18
```

- 并发 100，持续 3 秒

```
siege -c 100 -t 3s http://8.210.74.55:8002
```

```
Lifting the server siege...
Transactions:            679 hits
Availability:         100.00 %
Elapsed time:           3.57 secs
Data transferred:         0.02 MB
Response time:            0.48 secs
Transaction rate:       190.20 trans/sec
Throughput:           0.00 MB/sec
Concurrency:           91.39
Successful transactions:         679
Failed transactions:             0
Longest transaction:          0.68
Shortest transaction:         0.21
```

1.3.3 post

- 示例

```
curl --location 'http://8.210.74.55:8002/query' \
--header 'Content-Type: application/json' \
--data '{
    "query":"什么是保险"
}'
```

- 准备请求体文件 post_body.json

```
{
    "query":"什么是保险"
}
```

- 代码

```
siege -c 50 -t 10s -H "Content-Type:application/json" "http://8.210.74.55:8002/query POST <./post_body.json" >> report.txt
```

```
Transactions:           2673 hits
Availability:         100.00 %
Elapsed time:          10.82 secs
Data transferred:         0.27 MB
Response time:            0.20 secs
Transaction rate:       247.04 trans/sec
Throughput:           0.02 MB/sec
Concurrency:           49.60
Successful transactions:        2673
Failed transactions:             0
Longest transaction:          0.33
Shortest transaction:         0.12
```

1.4 输出解释

- 结果解释

字段	值	释义
Transactions:	2673hits	处理次数，本次共处理了 2673 次请求
Availability:	100.00%	请求成功率
Elapsed time:	10.82secs	测试总时长
Data transferred:	0.27MB	传输的数据总量
Response time:	0.20secs	平均响应时间
Transaction rate:	247.04trans/sec	每秒完成的请求数(TPS)
Throughput:	0.02MB/sec	每秒传输的数据量
Concurrency:	49.6	平均并发用户数
Successful transactions:	2673	成功的请求总数
Failed transactions:	0	失败的请求总数
Longest transaction:	0.33	最长请求时间
Shortest transaction:	0.12	最短请求时间

2 locust

2.1 安装

- **官方文档**

> https://docs.locust.io/en/2.29.0/writing-a-locustfile.html

以 `2.29` 版本为例子，若是 `2.24` 版本，则将上述 url中 `2.29.0` 改为 `2.24.0`

- pip 直接安装

```
pip install locust
```

- 查看版本

```
locust -V
```

2.2 重要参数

- **官方参数解释**

> https://docs.locust.io/en/2.29.0/configuration.html

- 查看参数

```
locust -h
```

- 参数说明

参数	参数说明
-h, --help	帮助，打印此部分。
-V, --version	版本，打印版本号。
-f, --locustfile	指定 Locust 文件。
--headless	无头模式，不启动 Web 界面。
-u, --users	并发用户数，默认值为 1。
-r, --spawn-rate	用户生成速率，默认值为 1。
-t, --run-time	运行时间，例如 10m 表示 10 分钟。

-H, --host	目标主机 URL。
--web-host	Web 界面的主机地址，默认值为空。
--web-port	Web 界面的端口号，默认值为 8089。
--csv	将结果保存到 CSV 文件。
--csv-full-history	将每个用户的完整历史记录保存到 CSV 文件。
--logfile	指定日志文件。
--loglevel	日志级别，默认值为 INFO。
--print-stats	在控制台打印统计信息。
--only-summary	只打印摘要统计信息。
--reset-stats	在测试开始时重置统计信息。
--html	将结果保存为 HTML 文件。
--skip-log-setup	跳过日志设置。
--step-load	启用步进加载模式。
--step-users	步进加载模式下的用户数。
--step-time	步进加载模式下的时间间隔。
--list	列出所有 Locust 任务。
--show-task-ratio	显示任务比率。
--show-task-ratio-json	以 JSON 格式显示任务比率。
--tags	只运行带有指定标签的任务。
--exclude-tags	排除带有指定标签的任务。
--config	指定配置文件。

2.3 使用

2.3.1 UI 启动

- 示例代码simple.py

```python
import locust

class MyUser(locust.HttpUser): # 创建 locust.HttpUser 子类
    wait_time = locust.between(1,2) #  每个 task 间隔 1-2 秒

    @locust.task
    def test_api(self):
        resp = self.client.get("http://8.210.74.55:8002")
        assert resp.status_code == 200
```

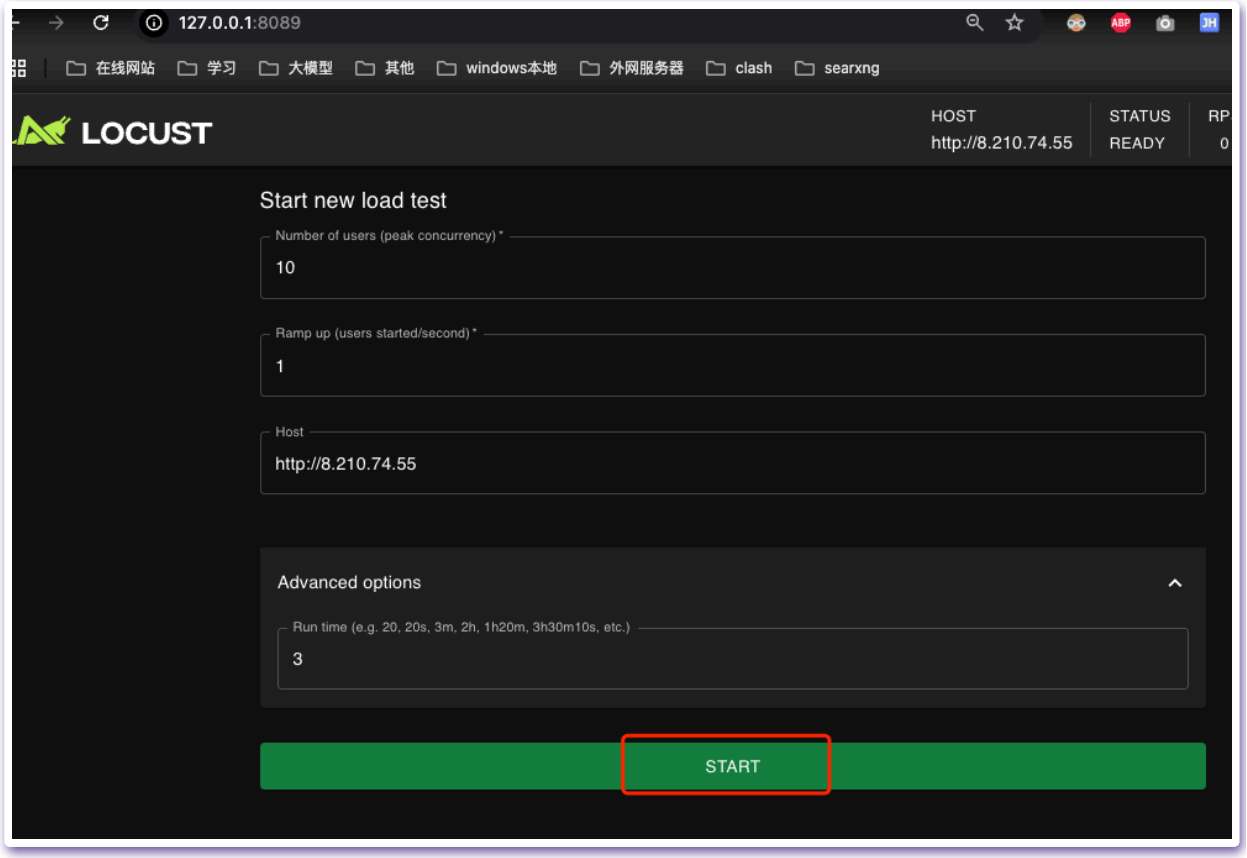
- 启动 Locust Web 界面

```
locust -f simple.py -u 10 -r 1 -H 'http://8.210.74.55' -t 30s
```

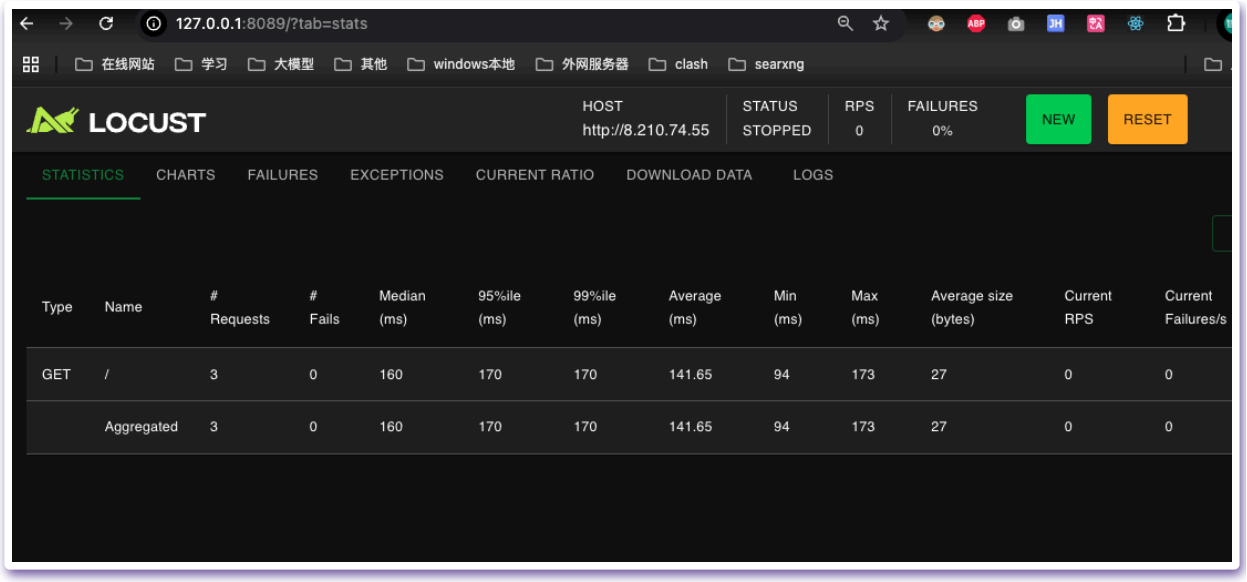
备注：

1. `-H` 后面跟的是host，这里已经在代码中指定了，所以随便写个111也行
2. `-f` 指定python文件
3. `-u` 指定并发用户数
4. `-r` 指定用户速率
5. `-t` 指定运行时间

- 启动 Locust 后，打开浏览器访问 `http://localhost:8089`，可以看到以下界面:



- 点击start即可运行

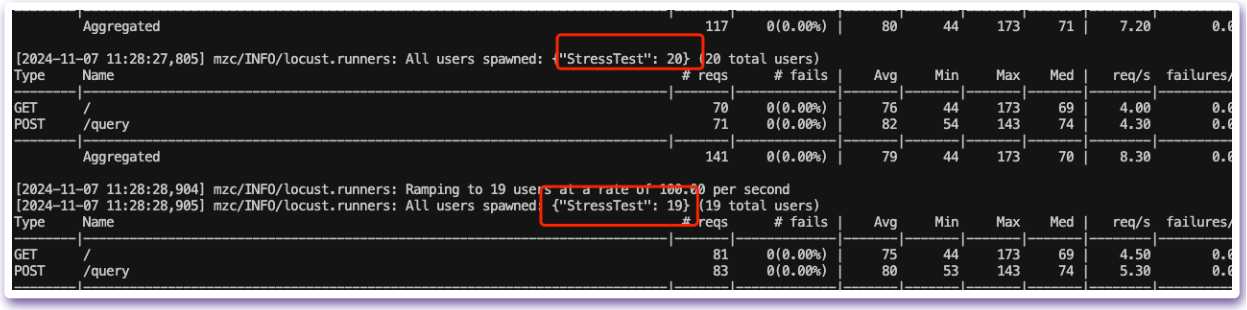


2.3.2 命令行启动

- 重要

> 在无头模式下，可以在测试运行时更改用户计数(并发数)。按 `w` 添加 1 个用户或 `W` 添加 10 个用户。按 `s` 删除 1 个用户或按 `S` 删除 10 个用户。

如下，在无头模式下按 `s` 删除了一个用户



所以注意无头模式如非需要，不要按到 `s,w` 键

- 非ui模型（无头模式）

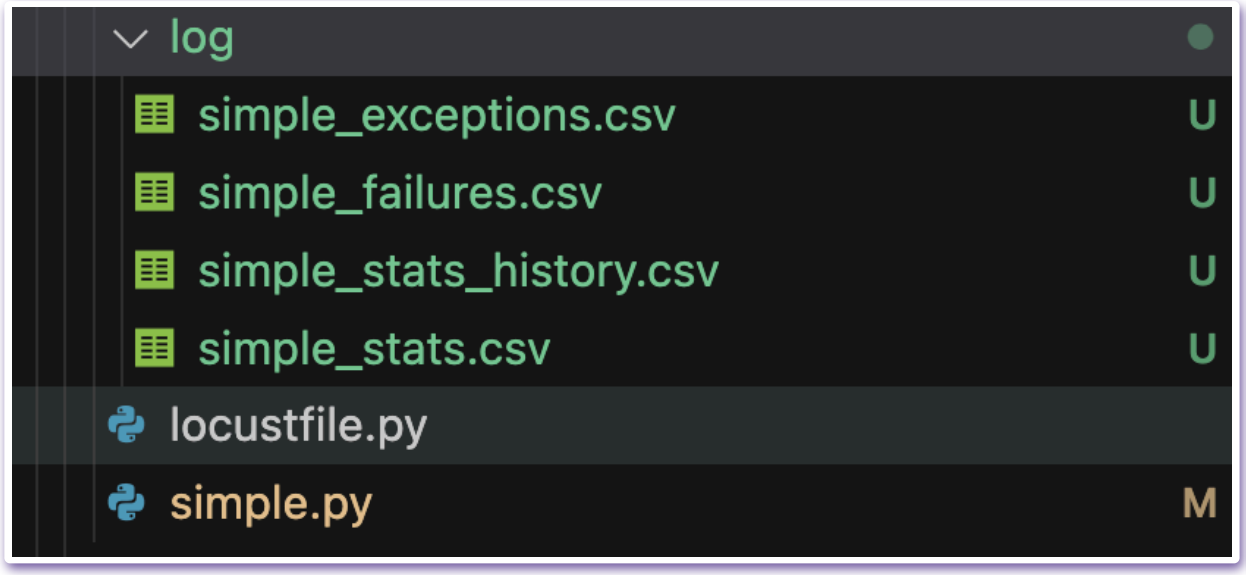
```
locust -f simple.py --headless -u 10 -r 1 -H 'http://8.210.74.55' -t 3s --csv ./log/simple
```

备注：

1. `-H` 后面跟的是host，这里已经在代码中指定了，所以随便写个111也行
2. `-f` 指定python文件
3. `-u` 指定并发用户数
4. `-r` 指定用户速率
5. `-t` 指定运行时间
6. `--headless` 无头模式，不启动 Web 界面。

7. `--csv` 结果保存为 csv 文件

- 输出文件



2.3.3 py文件启动

- 代码（locustfile_test.py）

```
import locust
import os

class StressTest(locust.HttpUser):
    # 设置用户等待时间，模拟用户在请求之间的停留时间
    wait_time = locust.between(1, 2) # 每个task间隔 1-2 秒


    # 初始化方法，相当于 setup
    def on_start(self):
        print("***开始压测***")

    @locust.tag('method_get')
    @locust.task
    def test_get(self):
        #测试get 接口
        resp = self.client.get("http://8.210.74.55:8002")
        assert resp.status_code == 200

    @locust.tag('method_post')
    @locust.task
    def tes_post(self):
        #测试post 接口
        payload = {"query": "什么是保险"}
        post_url = "http://8.210.74.55:8002/query"
        # 发送 POST 请求
        with self.client.post(post_url, json=payload, catch_response=True) as response:
```

```
            # 检查响应状态码
            if response.status_code != 200:
                response.failure(f"Failed with status code: {response.status_code}")
            else:
                response.success()

    # 结束方法，  相当于teardown
        def on_stop(self):
            print("***压测结束***")

if __name__ == "__main__":
    file_path = os.path.abspath(__file__)
    execute_task = 'method_post' # 指定指定任务
    os.system(f"locust -f {file_path} --headless -u 10 -r 1 -t \
              1m -H 111 --csv ./log/test_{execute_task} --tags {execute_task}")
```

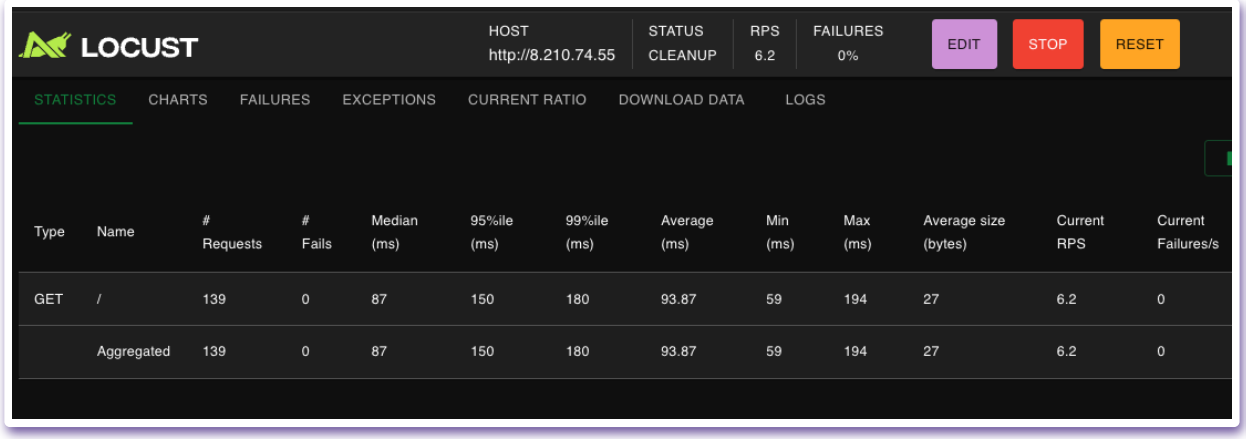
```
python locustfile_test.py
```

- 备注
    1. `@locust.tag` 任务标签
    2. `@locust.task` 装饰器将下面的方法标记为任务
    3. `-H` 后面跟的是host，这里已经在代码中指定了，所以随便写个111也行
    4. `-f` 指定python文件
    5. `-u` 指定并发用户数
    6. `-r` 指定用户速率
    7. `-t` 指定运行时间
    8. `--headless` 无头模式，不启动 Web 界面。
    9. `--csv` 结果保存为 csv 文件
    10. `--tags` 指定要运行的任务标签，这里由上面的 `@locust.tag` 标记了两个任务 `"method_get"` 和 `"method_post"`

2.4 web输出解释

2.4.1 STATISTICS

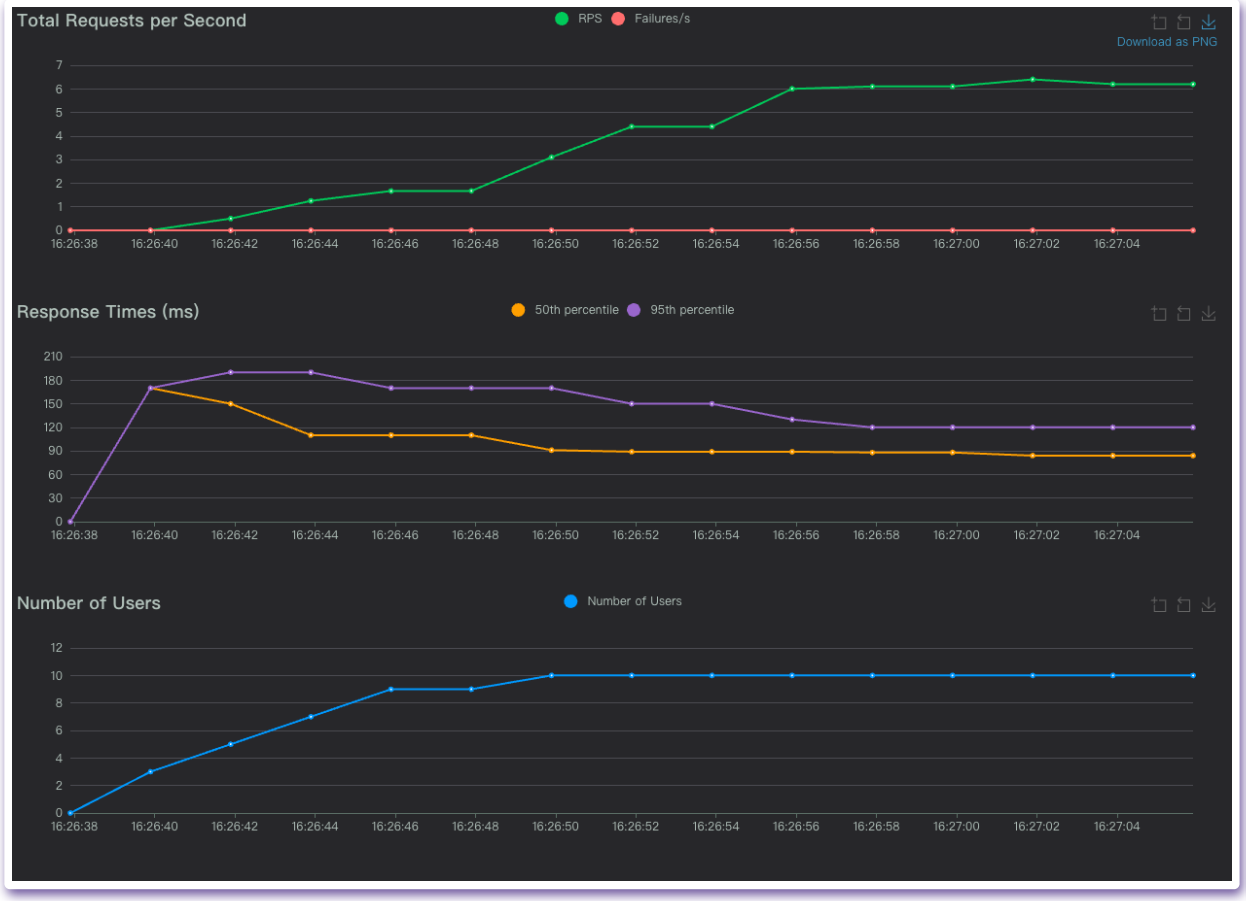
- web界面STATISTICS



字段	含义
Type	请求类型（GET、POST 等）。
Name	请求名称或路径。
Requests	总请求数。
Failures	失败请求数。
Median response time	中位数响应时间（毫秒）。
Average response time	平均响应时间（毫秒）。
Min response time	最小响应时间（毫秒）。
Max response time	最大响应时间（毫秒）。
Average Content Size	平均内容大小（字节）。
Requests/s	每秒请求数（RPS）。

2.4.2 CHARTS

- 统计图（仅运行 30s用于展示）



图名	含义
Total Requests per Second	每秒总请求数。
Response Times (ms)	响应时间分布图。
Number of Users	并发用户数图。

2.5 搜索场景

- 如果需要测试搜索场景或者测试大模型的 chat，需要不同的 query 或者 prompt，则可以使用以下代码(locustfile_cycle)

```
import locust
import os
from itertools import cycle
import pandas as pd
import threading
import json
import logging

# 配置日志记录
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

```

```python
# 创建异步锁，用于同步对文件的访问
lock = threading.Lock()

execute_task = 'method_post' # 指定指定任务
output_path =f'./save/{execute_task}_output.json'

df = pd.read_excel('./data/cycle_query.xlsx')
query_list = df['question'].values.tolist()
cycle_query = cycle(query_list)

def write_to_json(filename:str,data:dict):
    with lock:
        # 临界区，只有一个协程可以进入
        try:
            # 检查文件是否存在，如果不存在则创建它
            if not os.path.exists(filename):
                with open(filename, mode='w', encoding="utf-8") as f:
                    f.write('')

            # 写入数据到文件
            with open(filename, mode='a', encoding="utf-8") as f:
                f.write(f'{json.dumps(data,ensure_ascii=False)}\n')

        except Exception as e:
            logger.error(f"Failed to write data to {filename}: {e}")

class StressTest(locust.HttpUser):
    # 设置用户等待时间，模拟用户在请求之间的停留时间
    wait_time = locust.between(1, 2) # 每个task间隔 1-2 秒

    # 初始化方法，相当于 setup
    def on_start(self):
        print("***开始压测***")

    @locust.tag('method_get')
    @locust.task
    def test_get(self):
        #测试get 接口
        resp = self.client.get("http://8.210.74.55:8002")
        assert resp.status_code == 200

    @locust.tag('method_post')
    @locust.task
    def tes_post(self):
        #测试post 接口
        query = next(cycle_query)
        payload = {"query": query}
        post_url = "http://8.210.74.55:8002/query"
        # 发送 POST 请求
```

```
        with self.client.post(post_url, json=payload, catch_response=True) as response:
            # 检查响应状态码
            if response.status_code != 200:
                response.failure(f"Failed with status code: {response.status_code}")
            else:
                response.success()
                write_to_json(filename=output_path,data={"input":payload,"output":response.json()})


    # 结束方法， 相当于teardown
        def on_stop(self):
            print("***压测结束***")

if __name__ == "__main__":
    file_path = os.path.abspath(__file__)

    os.system(f"locust -f {file_path} --headless -u 10 -r 1 -t \
              3s -H 111 --csv ./log/test_{execute_task} --tags {execute_task}")
```

3 evalscope

- evalscope github地址

> https://github.com/modelscope/evalscope

3.1 安装

- 使用conda创建一个虚拟环境(也可以直接在现有的环境上)

```
conda create -n evalscope python=3.10 -y

conda activate evalscope
```

- pip安装依赖

```
pip install evalscope

pip install 'evalscope[perf]'
```

evalscope[perf] 是evalscope库中压测大模型的部分

3.2 重要参数

- evalscope库压测大模型参数解释

参数名	解释
url	API请求的目标地址
parallel	并发数
model	模型名称
number	压测样本数
api	使用的API类型，这里采用OpenAI兼容的API格式
dataset	测试使用的数据集类型
stream	是否使用流式响应，设置为False表示使用非流式响应
debug	是否开启调试模式，设置为False表示不开启
headers	HTTP请求头信息，可以包含API密钥用于认证
connect_timeout	连接超时时间，单位为秒
read_timeout	读取超时时间，单位为秒
max_tokens	生成回复的最大token数量限制

3.3 使用

- 以压测 openai 峰风格的为例，这里压一下硅基流动的，curl命令如下

```
curl --location 'https://api.siliconflow.cn/v1/chat/completions' \
--header 'Authorization: Bearer your_api_key' \
--data '{
    "model": "Qwen/Qwen2.5-72B-Instruct",
    "messages": [
        {
            "role": "system",
            "content": "你是一个助手"
        },
        {
            "role": "user",
            "content": "你好"
        }
    ],
    "max_tokens": 512,
    "stream": true
}'
```

- 针对上面的culr命令，压测代码如下 (需要将your_api_key更换为自己的)

```
from evalscope.perf.main import run_perf_benchmark

```

```python

def run_perf(parallel):
    task_cfg = {
        'url': 'https://api.siliconflow.cn/v1/chat/completions',
        'parallel': parallel,
        'model': 'Qwen/Qwen2.5-72B-Instruct',
        'number': 20,
        'api': 'openai',
        'dataset': 'openqa',
        'stream': False,
        'debug': False,
        'headers': {
            'Authorization': 'Bearer your_api_key'
        },
        'connect_timeout': 6000,
        'read_timeout': 6000,
        'max_tokens': 512
    }
    run_perf_benchmark(task_cfg)



run_perf(parallel=1)
```

有一个点需要注意点，如果自己部署的，需要压大模型首帧时间（首个token出现），则需要将stream设为True,否则非流式的一次性出来，首帧即尾帧

3.4 输出解释

- 会输出子啊output文件夹下，共5个文件

  其中主要的是这两个文件夹benchmark_summary.json 和benchmark_percentile.json

- benchmark_summary.json

```json
{
    "Time taken for tests (s)": 203.0674,
    "Number of concurrency": 1,
    "Total requests": 20,
    "Succeed requests": 20,
    "Failed requests": 0,
    "Output token throughput (tok/s)": 31.0685,
    "Total token throughput (tok/s)": 36.0176,
    "Request throughput (req/s)": 0.0985,
    "Average latency (s)": 10.1525,
    "Average time to first token (s)": 10.1525,
    "Average time per output token (s)": 0.0328,
    "Average input tokens per request": 50.25,
    "Average output tokens per request": 315.45,
```

```
    "Average package latency (s)": 10.1525,
    "Average package per request": 1.0,
    "Expected number of requests": 20,
    "Result DB path": "./outputs/20250430_230534/Qwen2.5-72B-Instruct/benchmark_data.db"
}
```

英文名称	含义
Time taken for tests (s)	测试总时长（秒）
Number of concurrency	并发数
Total requests	测试过程中发送的请求总数
Succeed requests	成功完成的请求数量
Failed requests	失败的请求数量
Output token throughput (tok/s)	每秒输出tokens
Total token throughput (tok/s)	每秒总tokens
Request throughput (req/s)	这个就是pqs
Average latency	从发送请求到接收完整响应的平均时间
Average time to first token	从发送请求到接收第一个token的平均时间 因为设置了stream=False,所以这里只有一帧
Average input tokens per request	每个请求的平均输入长度
Average output tokens per request	每个请求的平均输出长度
Average time per output token	生成每个token的平均时间
Average package per request	流式输出模式下，每个请求平均接收的数据包数量
Average package latency	流式输出模式下，接收每个数据包的平均延迟
Result DB path	这个是每条样本的请求信息，可使用sqlite3来访问

- benchmark_percentile.json

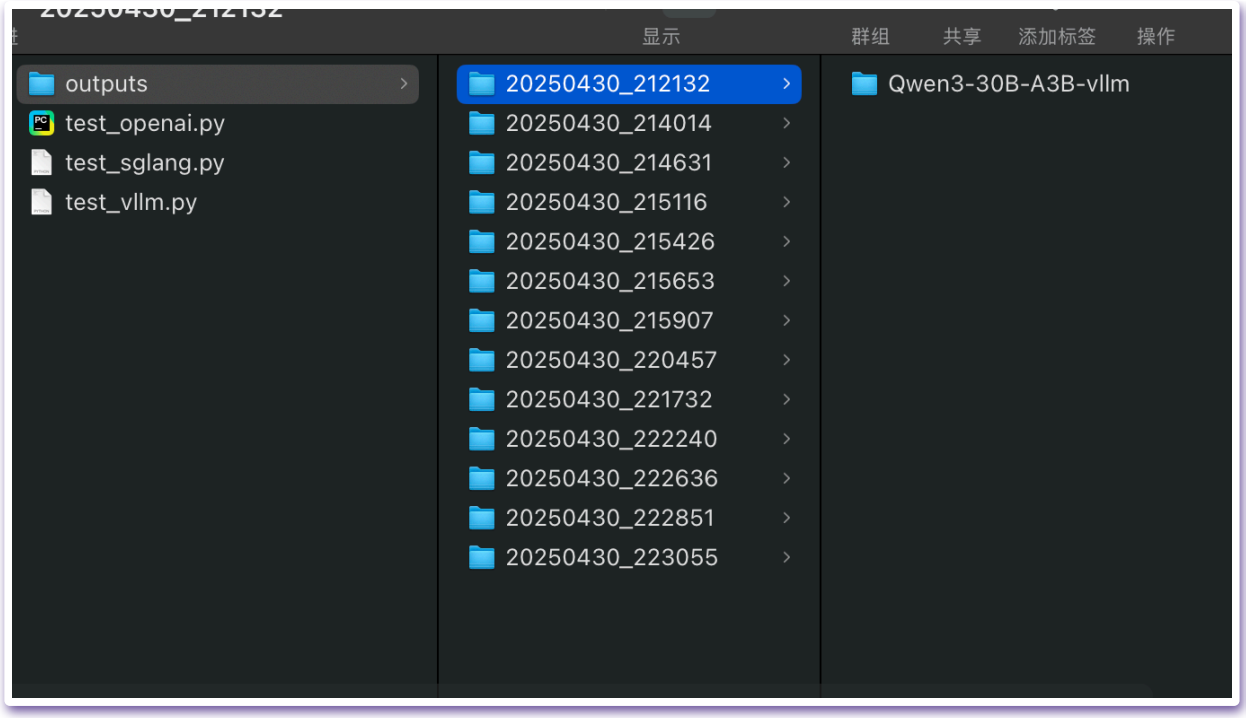
```
[
    {
        "Percentile": "10%",
        "TTFT (s)": 5.4016,
        "ITL (s)": NaN,
        "TPOT (s)": 0.0298,
```

```
            "Latency (s)": 5.4016,
            "Input tokens": 42,
            "Output tokens": 167,
            "Output throughput(tok/s)": 28.0987,
            "Total throughput(tok/s)": 32.7871
        },
    ......
    ]
```

英文名称	含义
`Percentile`	百分位数，表示数据分布中某个百分比的位置，例如 10%、25%、50% 等。
`TTFT (s)`	Time to First Token，生成第一个输出 token 所需的时间（秒）。
`ITL (s)`	Inference Time per Layer，每层推理所需的时间（秒）。文件中为 `NaN`。
`TPOT (s)`	Time per Output Token，生成每个输出 token 所需的时间（秒）。
`Latency (s)`	延迟时间（秒），当stream=False的时候等于 `TTFT (s)`。
`Input tokens`	输入的 token 数量。
`Output tokens`	输出的 token 数量。
`Output throughput(tok/s)`	输出吞吐量，表示每秒生成的输出 token 数量。
`Total throughput(tok/s)`	总吞吐量，表示每秒处理的 token 总数（输入和输出 token 的总和）。

3.5 解析代码

- 如果压测的时候有不同的模型需要压测，或者需要压测不同的并发数，可以使用for循环来实现，所有的结果都保存在outputs文件夹中



可以使用下面的解析代码来整合

```
import os
import pandas as pd
import json

def load_benchmark_data(outputs_dir):
    """
    加载 outputs 目录下的 benchmark 数据，并返回两个 DataFrame：
    1. benchmark_percentile_df：包含所有 benchmark_percentile.json 的数据，并添加 Number of concurrency。
    2. benchmark_summary_df：包含所有 benchmark_summary.json 的数据。

    :param outputs_dir: outputs 目录路径
    :return: (benchmark_percentile_df, benchmark_summary_df)
    """
    percentile_data = []
    summary_data = []

    # 遍历 outputs 目录
    for time_dir in os.listdir(outputs_dir):
        time_path = os.path.join(outputs_dir, time_dir)
        if not os.path.isdir(time_path):
            continue

        for model_name in os.listdir(time_path):
            model_path = os.path.join(time_path, model_name)
            if not os.path.isdir(model_path):
                continue

            # 定位 benchmark_percentile.json 和 benchmark_summary.json
```

```python
            percentile_file = os.path.join(model_path, "benchmark_percentile.json")
            summary_file = os.path.join(model_path, "benchmark_summary.json")

            if os.path.exists(percentile_file) and os.path.exists(summary_file):
                # 加载 benchmark_summary.json
                with open(summary_file, "r", encoding="utf-8") as f:
                    summary = json.load(f)
                    summary_data.append({
                        "time": time_dir,
                        "model_name": model_name,
                        **summary
                    })

                # 加载 benchmark_percentile.json
                with open(percentile_file, "r", encoding="utf-8") as f:
                    percentiles = json.load(f)
                    for percentile in percentiles:
                        percentile_data.append({
                            "time": time_dir,
                            "model_name": model_name,
                            **percentile,
                            "Number of concurrency": summary.get("Number of concurrency", None)
                        })

    # 转换为 DataFrame
    benchmark_percentile_df = pd.DataFrame(percentile_data)
    benchmark_summary_df = pd.DataFrame(summary_data)

    return benchmark_percentile_df, benchmark_summary_df

# 示例调用
outputs_dir = "c:/Users/孟智超/Desktop/eval_scope解析/outputs"
percentile_df, summary_df = load_benchmark_data(outputs_dir)

# 打印结果
print(percentile_df.head())
print(summary_df.head())
```